INTERNET OF THINGS

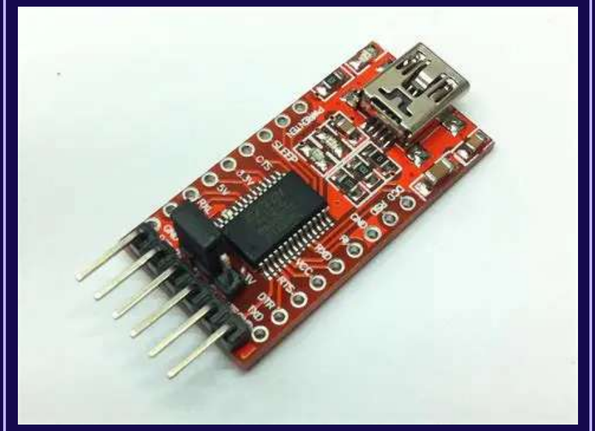
(IOT)

Billa Cherish	230150007
Harshit Beniwal	230150012
Yash Kharangate	230150032
Anant Sharma	230150034

VISION & CONTROL GATEWAY

ESP32-CAM & FTDI Interface Overview

- The Vision Core (OV2640): A 2-megapixel sensor optimized for Grayscale CIF resolution, reducing data overhead to ensure a high-speed video stream for real-time waste detection.
- The Processor (ESP32): A dual-core 240MHz "brain" that handles image capture and hosts a local Web Server, broadcasting live frames over Wi-Fi for model inference.
- The Bridge (FTDI/USB-to-TTL Adapter): Since the ESP32-CAM lacks a built-in USB port, this adapter acts as the essential communication link for flashing firmware and debugging via Serial Monitor.
- Power & Programming Logic: The adapter provides a steady 5V/3.3V power supply and manages the GPIO 0 to GND connection required to put the device into "Flash Mode" during code upload.
- Smart Mess Deployment: Together, these components create a modular edge device that monitors food waste "Prop" items and student inflow, feeding raw visual data into the LSTM predictive pipeline.



WEIGHT SENSOR

M5 Miniscales Unit and Working Overview

- The Hardware Core: An integrated 5kg load cell featuring a 24-bit HX711 Analog-to-Digital Converter for high-precision voltage reading, paired with an onboard STM32 ARM Cortex-M0 microcontroller to process raw analog signals locally.

- The Weighing Mechanism: As weight is applied, the internal cantilever beam undergoes microscopic bending. This physical deformation stretches onboard strain gauges, altering their electrical resistance within a Wheatstone bridge circuit to produce a measurable millivolt shift.

- The Data Pipeline: The onboard STM32 formats the weight data and transmits it to the main ESP32 via a two-wire I2C protocol. The ESP32 then packages this data into JSON and broadcasts it over Wi-Fi via HTTP POST requests to the Flask middleware for real-time UI updates.

ML PART ...

MODEL ARCHITECTURE

```
model = models.Sequential([
    layers.LSTM(64, activation='tanh', input_shape=(6, 10), return_sequences=True),
    layers.LSTM(32, activation='tanh'),
    layers.Dense(32, activation='relu'),
    layers.Dense(4, activation='linear')
])
```

LOSS FUNCTION

$$L(y, \hat{y}) = \begin{cases} 2(y-\hat{y})^2 & \text{if } (\hat{y} > y) \\ (y-\hat{y})^2 & \text{if } \hat{y} \leq y \end{cases}$$

WHY THIS !!!!

Mess traffic is a dynamic flow. LSTMs capture the 1-minute temporal trajectory, unlike standard DNNs which treat events as static snapshots.
Also more penalty should be given when food is overcooked as compared to when it is undercooked

Training assumptions

- Cook will prepare food for next batch(i.e next 30 min).
- A 1-minute window of sensor history is used to predict a 30-minute future kitchen batch.
- On average nearly 60 students are expected to come in 30 min slot.
- Waste generation is mathematically tied to dish popularity scores (Lower Score = Higher Waste Probability).
- Recommended Batch = Expected Demand - Penalty for Current Waste

INTEGRATION

Logging Sensor readings

- **ESP32-CAM:** The ESP32-CAM streams the live feed of the camera on its own ip address. We are using the OpenCV library to view these frames, count the no. of people entering, and sends the count to the node.js server.

- **HX711-Load:** We have established a flask server, wich receives the readings from the ESP32 Core 2 device via HTTP and TCP Protocol. We then send the data from the Flask server to node.js server.

- **Database:** The above readings get sent to MongoDB, where we have created our database, from node.js server

- **LSTM:** We are storing the last 6 sensor readings using a sliding window in the node.js server, and we are posting them to flask server where our LSTM takes the inputs and gives the predictions as the response to the post request.

USER INTERFACE

Mess Manager‘s Dashboard

- **Control Panel:** The mess manager can enter how much food has already been prepared for the current batch.
- **Live Feed:** The dashboard shows real time values of the aggregate amount of people who have entered the dining area and also the aggregate amount of waste that has been generated.
- **Projections:** How much food has to be made for the next batch so that the waste is minimized as much as possible.

Goal

- The mess manager can make use of the projections by ordering the chefs to only cook that much amount of food so that the **waste is minimized** for the upcoming batch.
- **Uneven Activity:** Whenever there is a **sudden spike** in the amount of people entering, or waste generated, the mess manager can simply use these intelligent projections to exactly give which food item the priority instead of going at the same pace

AMINGAON'S MICHELIN DINING

MANAGERS DESK

PEOPLE TRACKER

WASTE TRACKER

SHORTCOMINGS

- **Deployment:** We couldn't get time to install the sensors and track real time data. Hence we have used synthetically generated data.

FUTURE SCOPE

- **Individual Tracking:** We can install further cameras to track which food items and the individual quantities that are being taken.
- **Chef’s IoT:** We could give the head chef as well an IoT device to directly view the AI projections and also to make efficient communication with the mess manager.
- **Inter-mess communication:** We could as well design communication between all the messes in the campus to monitor which food items are being given the priority, sudden spike in activities, etc, and learn from them. It could further be used for designing a better menu.

THANK YOU